IOT SECURITY LAB

Assignment #3 — IoT Security Lab: Final Report (V2.2)

Evidence-backed written report · Frozen specification V2.2

Experiment	Final run	Verdict
E1 · ARP Spoofing / MITM	20261028T202401Z	PASS
E2 · Replay	20261028T202400Z	PASS
E3 · SQL Injection	20261028T202401Z	PASS

Final acceptance: **PASS** · Clean-room checks: 20 passed, 0 failed, 0 skipped

Canonical specification: docs/ASSIGNMENT3_IOT_SECURITY_LAB_V2_2_FROZEN.md

All reported values are reproduced from committed evidence under evidence/<experiment>/<run_id>.

Submission date: 30 October 2026

1. Introduction

IoT security is not device-only security. A connected device participates in a **trust chain** spanning Device → Network → Transport → Application → Data. A control at one layer protects one failure domain only; a gap at any other layer still breaks the system. This lab demonstrates that claim empirically with three attacks at three different trust boundaries, each with its own distinct attacker.

2. Assignment Requirement

- **Primary attack:** ARP Spoofing / MITM (E1).
- **Cross-layer extensions:** Replay (E2) and SQL Injection (E3).
- Fallback order (spec §3/§33): if E1 network feasibility FAILs → E3 becomes

primary → then E2. In this run E1 feasibility **PASSed**, so E1 is the primary experiment as intended.

3. Architecture

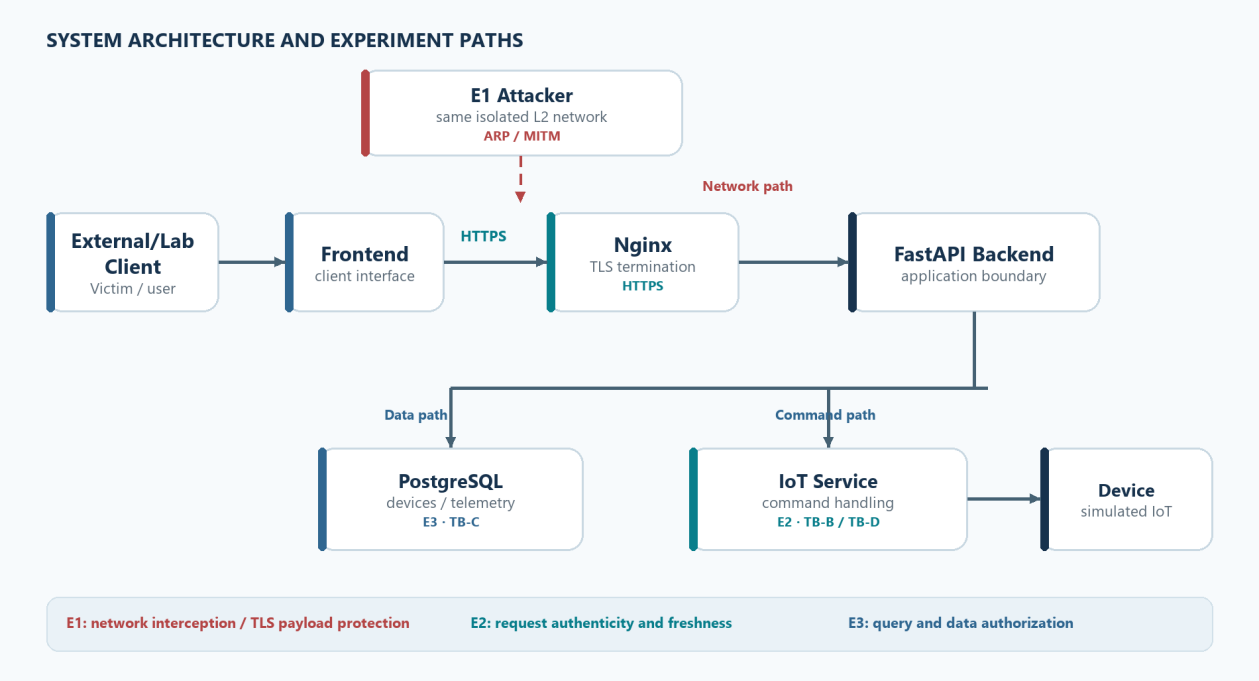


Figure 1. Application architecture and the E1, E2, and E3 experiment paths.

The database is **not** on every IoT command path. See `COMMON_BASELINE.md` for the mapping from this architecture to the runnable artifacts.

4. Threat Model

Exp	Attacker position	Capability	Explicit assumption
E1	same L2 network	ARP manipulation, interception	HTTP observable; correct TLS protects payload
E2	holds one captured legitimate request	replay / modify it	request from log leak / post-TLS internal; **TLS not broken**
E3	can reach Backend API	supply malicious input	request already reached Application layer

Three distinct attackers — not one omnipotent adversary.

5. Primary Experiment — ARP Spoofing / MITM (E1)

Table 1. E1 before/after evidence summary — final run 20261028T202401Z.

Check	Before / attack state	After control / recovery	Evidence
ARP mapping	Gateway 10.77.0.2 resolves to attacker MAC 32:83:f3:8c:36:6f.	After restore, gateway MAC 8e:2e:67:45:e0:fd is present again.	E1-01, E1-02, E1-08
Payload visibility	HTTP capture exposes the test token twice.	HTTPS capture has 0 token occurrences; the victim still receives a valid response.	E1-04, E1-05, E1-06
TLS validation and service	Without forwarding, traffic breaks (DoS); this alone is not MITM.	Trusted CA succeeds (exit 0); untrusted CA is rejected (exit 60, HTTP 000); normal HTTP/TLS works after restore.	E1-DOS, E1-07, E1-08

Environment (spec §2/§5). An isolated private Docker L2 network `e1_net` (`10.77.0.0/24`), three project-owned containers: victim `.10`, gateway `.2`, attacker `.66`. Not bridged to any real LAN; targets are fixed by the topology (scope §0).

5.1 Network Feasibility Gate (P0, spec §2.1) — PASS

Condition	Result
SAME_L2_DOMAIN	YES
ARP_TABLE_VISIBLE	YES
ARP_MAPPING_CAN_CHANGE	YES
ATTACKER_IP_FORWARDING	YES
TRAFFIC_CONTINUES_THROUGH_ATTACKER	YES
PCAP_EVIDENCE_AVAILABLE	YES

Evidence: `feasibility_gate.json`, `E1-01_baseline_arp.txt`, `E1-02_poisoned_arp.txt`.

5.2 Root cause (spec §7.1)

ARP has no authentication. The attacker sends unsolicited ARP replies binding the **gateway IP → attacker MAC** (and victim IP → attacker MAC at the gateway). The victim ARP entry for `10.77.0.2` changed from the gateway MAC (`8e:2e:67:45:e0:fd`) to the attacker MAC (`32:83:f3:8c:36:6f`). Evidence `E1-01` vs `E1-02`.

5.3 MITM vs DoS (spec §7.2)

- **Forwarding dropped** (ARP poisoned, attacker drops forwarded packets): victim request to the gateway **broke** (empty response). This is DoS/disruption, **not** MITM. Evidence `E1-DOS_http_response.txt`.
- **Forwarding enabled**: victim traffic flows victim → attacker → gateway and back; the victim still receives a valid response → **true MITM path**. Evidence `E1-05_mitm_http_response.txt`.

5.4 Confidentiality: HTTP vs HTTPS (spec §7.3, §10)

- **HTTP:** the plaintext token `SECRET_TELEMETRY_TOKEN_12345` appears in the attacker's capture (2 occurrences). Payload exposed. Evidence `E1-04_mitm_http_pcap.txt`, `E1-05`.
- **HTTPS (certificate validation ENABLED):** under the same MITM path, the token does **not** appear in the attacker's TLS capture (0 occurrences), while the victim still gets a valid response. Payload confidentiality preserved. Evidence `E1-06_mitm_https_pcap.txt`, `E1-06_https_response_validated.txt`.
- **Cert validation is genuinely enforced** (no `verify=false`): with the trusted CA the connection succeeds (curl exit 0); without the CA in the trust store the connection is **rejected** (curl exit 60, HTTP code 000). Evidence `E1-07_cert_validation.txt`.

5.5 CIA (spec §7.5)

Integrity: ARP mapping modified. Confidentiality: plaintext HTTP exposed, HTTPS protected. Availability: dropping forwarding degrades service (DoS case).

5.6 Defense & Regression (spec §7.6, §7.7, §13)

Implemented defense: **TLS + certificate validation**. After stopping the attack and restoring ARP, normal traffic works: HTTP OK, valid TLS OK, and the victim's ARP entry is restored to the gateway MAC — the control passes **without** relying on "block everything". Evidence `E1-08_restored_arp.txt`, `E1-REGRESSION`.

5.7 Residual risk (spec §7.8)

TLS protects payload confidentiality/integrity but not network availability or endpoint security. If an endpoint is compromised, plaintext is exposed before encryption / after decryption. DAI, segmentation, static ARP and ARP monitoring are **conceptual** here (not implemented).

6. Replay Extension (E2)

Table 2. E2 before/after evidence summary — final run 20261028T202400Z.

Check	Before control	After control	Evidence
First request	Accepted; action_count = 1.	Accepted once with HMAC over raw body, timestamp, and nonce.	result.json
Exact replay	Accepted again; action_count changes 1 → 2.	Rejected with HTTP 401 because the nonce was already consumed; action_count stays 1.	result.json, audit_log.json
Modified request	No request authentication in the vulnerable baseline.	Old signature with changed nonce, timestamp, or body is rejected for signature mismatch.	result.json, audit_log.json
Audit trail	No protection audit in the vulnerable baseline.	Five ACCEPT/REJECT events recorded for the protected cases.	audit_log.json

Boundary TB-B/TB-D. Attacker holds one captured legitimate request and does **not** break TLS (spec §9).

- **Vulnerable baseline:** no authentication → exact replay accepted → `action_count` 1 → 2. Evidence `E2-VULN-EXACT-REPLAY`.
- **Protected:** HMAC-SHA256 over the **raw received body bytes** + timestamp

window + nonce cache, compared with `hmac.compare_digest` (constant-time), per-device secret (spec §11–§18.1).

Gate	Result
FIRST_REQUEST	ACCEPT (action_count=1)
EXACT_REPLAY	REJECT (nonce already consumed)
MODIFIED_NONCE + old signature	REJECT (signature mismatch)
MODIFIED_TIMESTAMP + old signature	REJECT (signature mismatch)
MODIFIED_BODY + old signature	REJECT (signature mismatch)
ACTION_COUNT on replay	UNCHANGED (=1)
AUDIT_LOG_CREATED	YES (ACCEPT + REJECT events)

The canonicalization rule is FROZEN: the client signs the exact bytes it transmits; the server verifies the exact bytes it received — never JSON parse-then-reserialize (a unit test proves two byte-different but semantically-equal JSONs yield different signatures).

Replay ≠ Idempotency (spec §17). Replay protection (nonce = security freshness) is **IMPLEMENTED**; idempotency (`idempotency_key` = retry semantics) is **CONCEPTUAL / future work**.

Residual risk (spec §18/§19): compromised device secret defeats HMAC; in-memory nonce cache is lost on restart (previously-used nonce within the timestamp window could be re-accepted) — `E2_RESIDUAL_RISK`; clock drift. A global shared fleet secret is rejected in favor of per-device secrets; hardware key protection (TPM/SE/TEE) is conceptual only.

7. SQL Injection Extension (E3)

Table 3. E3 before/after evidence summary — final run 20261028T202401Z.

Check	Before control	After control	Evidence
Normal query	sensor-alpha returns 1 row.	Parameterized query still returns 1 row.	result.json
Injection semantics	Raw SQL input ' OR '1'='1 returns 4 rows instead of 1.	Same input returns 0 rows through the parameterized query.	E3-01, E3-02, result.json
Privilege boundary	Least-privilege controls are not part of the vulnerable SQL-interpolation baseline.	iot_search_reader can read allowed tables; sensitive read, UPDATE, and DROP are denied; database row count remains 4 → 4.	result.json

Boundary TB-C. Attacker reaches the Backend API and supplies input (spec §6).

- **Vulnerable baseline (genuine raw SQL interpolation, spec §21.1):** normal input `sensor-alpha` → 1 row; injection `' OR '1'='1` → 4 rows. Input changes query semantics. Evidence `E3-01`…`E3-02`, `E3-VULN-SEMANTICS`.
- **Fix — Parameterized query (PREVENT, spec §24):** the same injection input now returns 0 rows; normal input still returns 1. `SAME_INPUT_CHANGES_QUERY = NO`.
- **Containment — least-privilege role `iot_search_reader` (CONTAIN, spec §25–27):**

Gate	Result
Allowed-table SELECT (devices/telemetry)	permitted
SENSITIVE_TABLE_READ (UNION → users)	DENIED (InsufficientPrivilege)
UNAUTHORIZED_UPDATE	DENIED (InsufficientPrivilege)
UNAUTHORIZED_DROP	DENIED (must be owner)
DB state before/after	unchanged (4 = 4)
NORMAL_QUERY_REGRESSION	PASS

Least privilege limits blast radius but is **not** a root-cause fix — injection on an allowed table could still read all permitted telemetry (spec §27).

8. Cross-Layer Analysis



Figure 2. Cross-layer trust boundaries and their corresponding controls.

Each control protects exactly one failure domain. Correct TLS (E1) does nothing against a replayed authenticated request (E2); replay protection (E2) does nothing against malicious SQL from a legitimate client (E3); parameterized queries (E3) do nothing against L2 interception (E1). Hence defense must span layers.

9. Defense-in-Depth (spec §14/§29) — `[I]` implemented+verified, `[C]` conceptual

Exp	Prevent	Detect	Contain	Recover
E1 ARP/MITM	**[I] TLS + Cert Validation** / [C] DAI	[C] ARP anomaly monitoring	[C] Network segmentation	[C] Restore trusted network state
E2 Replay	**[I] HMAC + Nonce + Timestamp**	**[I] Invalid-sig / duplicate-nonce audit**	**[I] Reject request** / [C] revoke device key	[C] Key rotation / session recovery
E3 SQLi	**[I] Parameterized Query**	[C] DB/API anomaly monitoring	**[I] Table-level Least Privilege**	[C] DB restore / credential rotation

This lab implements **Prevent + partial Detect + Contain**. A complete Detect/Recover pipeline is **not** claimed.

10. Residual Risk (spec §32.10)

Endpoint compromise (pre-/post-encryption plaintext); HMAC key leakage; in-memory nonce-cache restart; clock drift; idempotency (conceptual); application-accessible SQL data still readable within granted privileges even after the injection fix.

11. Limitations

Controlled lab; simulated IoT device; no public target; no destructive operations; no Internet DDoS; no production credentials. E2 driven in-process for determinism (same application logic as the canonical FastAPI backend).

12. Conclusion

IoT security is a cross-layer trust-management problem. Each experiment confirms a single control secures a single failure domain: TLS protected the HTTPS payload but not availability; HMAC+nonce stopped replay but not a leaked key; parameterized queries removed the injection root cause while least privilege only contained blast radius. Therefore **defense-in-depth** across Network, Message, and Application/Data boundaries is required.

13. References

1. Course slides — IoT Application Information Security & Risk Management.
2. Assignment #3 slides (DVWA SQL Injection option).
3. RFC 826 — Address Resolution Protocol (ARP).
4. RFC 2104 — HMAC: Keyed-Hashing for Message Authentication.
5. RFC 8446 — TLS 1.3.
6. OWASP Top 10 — A03:2021 Injection.
7. OWASP — SQL Injection Prevention Cheat Sheet (parameterized queries).
8. PostgreSQL documentation — GRANT / role privileges (least privilege).
9. CWE-287 (Improper Authentication), CWE-294 (Replay), CWE-89 (SQL Injection).
10. Real-world context: Mirai botnet (2016) — IoT device trust failure at scale.